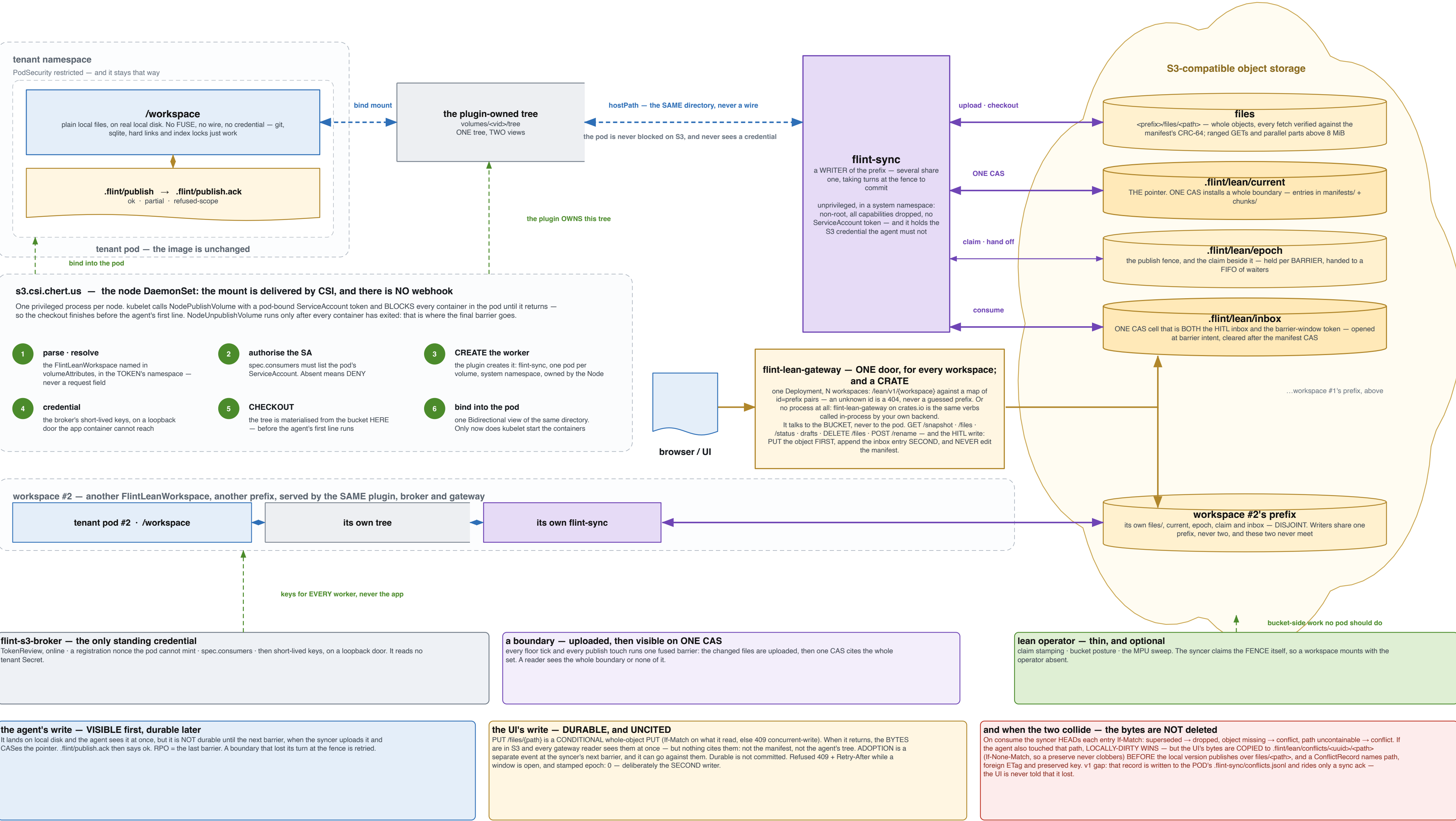


Data flow

Nothing sits between the pod and its files. A worker beside it — holding the credential the pod must not — reconciles the same directory with the bucket at a boundary the agent declares.



THE ORDERING IS THE ARCHITECTURE. kubelet blocks every container in the pod on NodePublishVolume, so the checkout completes before the agent's first line runs — no init container, no readiness dance — and calls NodeUnpublishVolume only after every container has exited, which is where the final barrier goes. In between, the app touches plain files on real local disk with zero interception.

THE MOUNT IS DELIVERED BY CSI, AND THERE IS NO WEBHOOK. The tenant pod declares a csi: volume naming a FlintLeanWorkspace in its own namespace and stays admissible under PodSecurity restricted: no injected sidecar, no mutating admission, no cert bootstrap, no privileged container and no S3 credential anywhere in the tenant namespace. Privilege is concentrated instead — the node plugin (one per node, holding no S3 credential and no Secrets RBAC), the worker (non-root, no ServiceAccount token) and the broker (the only standing credential, every issuance TokenReview-verified and audit-logged).

THE BOUNDARY VERBS ARE FILES, because the workspace is the interface. An agent that can write a file can declare a coherent point — echo > .flint/publish — and learn when its bytes are durable, from .flint/publish.ack: ok means the bytes are in S3, partial names what a foreign write kept out, and a boundary that lost its turn at the fence is retried rather than refused. No client library, no credential, no network path.

A UI IS POWERED BY THE BUCKET, NOT BY THE POD, and that is forced rather than chosen: the workspace tree is local disk that dies with the pod, and the pod may not exist at all, so there is nothing for a UI to call. flint-lean-gateway — opt-in, its own bearer, an explicit workspace map, and deliberately NOT the lite gateway — reads and writes the same CAS calls the syncer uses, and since 0.1.0 it is a CRATE too: a backend depends on flint-lean-gateway and calls the verbs in-process, holding nothing but credentials for the prefixes it serves. GET /snapshot returns (manifest, manifest_etag, inbox) in one read; GET /files/<path> reads guarded on the citation and consults the inbox only when that fails, so an overwrite reads at once and the common read costs nothing extra; delete and rename are DECLARED removals the syncer performs at its barrier (a rename is one generation; a removal of a dirty path is refused, never retried); drafts are durable edits nobody sees until promoted.

THE HITL WRITE IS THREE ORDERED STEPS AND THE MANIFEST IS NOT ONE OF THEM. PUT /files/<path> writes the OBJECT first and appends an INBOX entry second — never a manifest edit — so only syncers write the pointer, one commit at a time. A worker consumes the inbox at its next barrier (HEAD each entry If-Match; a superseded entry is dropped, not an error), materialises it into the tree and cites it in the next manifest. A write is refused 409 + Retry-After while a barrier window is open, and every gateway replica reads that window from the CELL rather than from its own memory — which is what makes two stateless replicas safe.

THE WINDOW CANNOT WEDGE, AND A DEPOSED WORKER CANNOT WRITE. The window carries a deadline and a successor epoch may override a stale one, so a dead worker does not block HITL forever; and every worker-facing verb — window/open, window/clear, inbox/drop, manifest — is epoch-validated PER REQUEST, so a write whose claimed epoch is not the cell's current epoch is rejected. Rotation alone leaves that door open, which is exactly what the model's LeanNoEpochCheck mutation proves.

AND ONE VERB IS DELIBERATELY CARRIED, NEVER PERFORMED. "Please publish" from outside the pod is honoured; "please pull" is recorded and left to the agent. A boundary publishes what is already on disk and touches no local file, while a sync re-derives the tree and DELETES local files for remotely-deleted paths — so performing it on a remote's say-so would upgrade what a leaked gateway bearer can do from "publish, plus hand over these N named objects" to "rewrite and delete across a running agent's tree, at my timing, under a scope I choose". v1's recorded limits: whole-object HITL writes under a cap, and one shared bearer.

NOTHING HERE IS INJECTED, AND THERE IS NO WEBHOOK IN ANY OF IT. The flint-sync worker is CREATED by the node plugin during NodePublishVolume — one pod per published volume, in a system namespace, pinned with nodeName so it skips the scheduler, and owned by the Node object so a vanished node garbage-collects it. It is not a container in the tenant's pod and no mutating admission rewrites the tenant's spec: the component is the SYNCER, and flint-sync is its binary. The gateway, the broker and the operator are not injected either — each is an ordinary Deployment installed by Helm, and the gateway is opt-in: its chart REFUSES to render without a token Secret and a workspace map, because an unauthenticated gateway is an open writer to every workspace configured.

WHY THE GATEWAY APPENDS TO AN INBOX AND NEVER CASES THE MANIFEST. Not because a second CAS writer would corrupt it — CAS would arbitrate that perfectly well. Because a manifest write is not a metadata edit, it is a CLAIM ABOUT A TREE: a boundary means "everything ordered-before T", and the entries it cites are one half of a pair whose other half is the syncer's on-disk baseline — classify() derives uploads from "changed vs the baseline" and deletes from "absent twice AND present in the baseline". The gateway has no tree (its root is literally /nonexistent), no baseline and no scan, so it cannot honestly say what a coherent boundary contains. It CAN honestly say "these bytes exist and somebody asked for them", and that is exactly what an inbox entry is: a proposal about content, not a claim about a tree.

CONCURRENT UI WRITERS ARE HANDLED IN TWO PLACES AND MISSED IN A THIRD. The inbox cell is CAS'd with a five-attempt retry, so two browsers appending at once serialise and the loser retries — past five it fails the request rather than dropping the entry. And the queue holds ONE entry per path: a newer write supersedes the queued one, and at consume an entry whose ETag no longer matches the object is dropped as superseded. So one path yields one adoption of the last write, however many browsers raced. The object PUT is conditional as well — but on a FRESH HEAD the gateway takes immediately before writing, which closes the HEAD-to-PUT window and is NOT end-to-end optimistic concurrency: so an overwrite must now NAME what it read: 428 precondition-required without an If-Match, 412 file-changed on a stale one, and the current etag on the 412's own header. That is forge's taxonomy, adopted so the three doors are one shape. Until this landed the route accepted no caller If-Match at all, and two browsers that each read v1 and then wrote BOTH succeeded — the second silently winning.

AND WHY NOT SIMPLY ROUTE TO THE WORKER, the way lite's gateway routes to a hub? Because half the time there is nothing to route TO. A lean worker is created by the node plugin at NodePublishVolume and DELETED at NodeUnpublish after the drain, so a workspace no tenant pod is mounting has no flint-sync anywhere — it is only a prefix in a bucket, and a UI must still be able to list and write it. A lite hub is a Deployment an operator can scale 0→1 and wake; a lean worker is a side effect of a POD being scheduled, and no gateway can conjure a tenant pod. Nor is there a door to route to: flint-sync's control surface is a UDS in the pod's emptyDir, and the code says why it stays that way — "there is no TCP listener and no authentication, because the trust boundary is the pod; a TCP listener would be a new remote surface, which is a stated non-goal".

AND ROUTING WOULD HAND THE UI THE WRONG GUARANTEE, which is the part that surprises. A write placed into the worker's tree is visible at once and durable only at the next barrier — the AGENT's bargain. The UI's 200 would then precede the bytes reaching S3, and a pod that dies in between loses the write. The inbox inverts exactly that: durable when the call returns, adopted later. The same line is already drawn inside the pod — the UDS sync verb EXECUTES, "because the caller is inside the pod: it is the agent asking for its own tree to be updated, which is the agent's own decision to make", while the gateway's sync request is carried and never performed. Inside the pod is a decision; outside it is a proposal.

AND THREE CONSEQUENCES FOLLOW FROM THAT ONE. A manifest write would need the fence the gateway does not hold — it stamps epoch: 0 — so either the gateway queues for the fence like a syncer and runs a merge, or manifest writes stop being fenced, which is the deposed-straggler hole. It would move the pointer's ETag on every UI write, so baseline.manifest_etag would no longer match and the syncer would take its adopt-a-foreign-manifest path on every one of them — an exceptional path made the common path. And it would read-modify-write an entries set that runs to hundreds of MiB at a million files, racing the barrier's own commit. The inbox is a few hundred bytes, needs agreement with nobody, and doubles as the barrier-window token, so "is a barrier in flight?" and "queue this write" are one read and one CAS.

THE GATEWAY IS A WRITER, BUT NOT THE SAME KIND OF WRITER, and the two guarantees are exact opposites: the agent's write is visible before it is durable, the UI's is durable before it is visible. What keeps that safe is that only the SYNCERS write the manifest, one commit at a time. The gateway stamps epoch: 0 and never edits the pointer; a syncer holds the fence for its commit and cites the inbox at its own barrier. So "written" means different things at the two doors, and a UI that reports success on a PUT is reporting durability, never adoption.

TWO WORKSPACES SHARE THE ARCHITECTURE AND NOTHING ELSE. The node plugin (one per node, the broker is one Deployment, and the gateway is one process whose workspace map is a list of id-prefix pairs — an unknown id is a 404, never a guessed prefix, and in v1 ONE bearer covers every workspace in that map, which is the recorded limit to know before pointing a UI at a multi-tenant one. Everything below that is per volume: its own worker pod (one per PUBLISHED volume, created by the plugin on its own node and owned by the Node so a vanished node GCs it), its own tree, its own credential, and its own prefix with its own epoch, claim, pointer and inbox. Taking turns at one prefix's fence is a mechanism inside a product; across products it is only a convention, so what assigns prefixes is what keeps two workspaces apart.

THREE QUESTIONS DECIDE AGAINST LEAN, cheapest disqualifier first: does the tree fit the disk, and its file count the checkout budget (the manifest is chunked and a publish is O(changed), but every checkout still materialises every file); is a shared log with per-pod working copies enough (a same-file edit is the later boundary's, the other kept); is snapshot freshness acceptable. A no on any of them means the hub. And a worker is never taken away from a tenant still using its tree by ORDERING, not by a PodDisruptionBudget — a PriorityClass for kubelet's graceful shutdown and a preStop that waits for the release, because on a spot fleet the eviction API is not involved at all.

-----> the workspace — one directory, two views, never a wire

-----> durable path — only workers write the prefix, a commit at a time

-----> control plane — never carries a file

-----> the file protocol — boundary verbs in the tree, REST at the gateway

What the abbreviations mean

CSI ephemeral inline a volume declared in the pod spec itself — no PVC, no PV, nothing to bind beforehand	NodePublishVolume the CSI call kubelet BLOCKS every container on. The checkout happens inside it	NodeUnpublishVolume the call kubelet makes only after every container has exited — the final barrier	bind mount · hostPath two names for ONE directory: the plugin's tree, seen from the pod and from the worker
CAS compare-and-swap — a write that lands only if the object is still the version you read	ETag / If-Match S3's precondition: the ETag you read, offered back as the terms of the write	the pointer .flint/lean/current — the ONE mutable metadata object. Entries live in immutable manifests	fence whose turn it is to commit: held for one barrier's commit, then handed on. A deposed holder's late CAS is refused
epoch · lease the bucket-side commit cell: one holder, a FIFO of waiters. It lives in the BUCKET, so it fences a writer in ANY cluster	claim the project identity stamped on the prefix: adopt your own, refuse a foreign one	boundary verb a file the agent writes to declare a coherent point, and an ack file it reads back	boundary one fused barrier: upload the changed files, then install the whole set with one CAS
loopback door AWS_CONTAINER_CREDENTIALS_FULL_URI: how short-lived keys reach the worker, and only the worker	TokenReview the Kubernetes API that says whether a ServiceAccount token is still valid — online, not offline	FlintLeanWorkspace the custom resource one lean project is declared as, in the tenant's own namespace	RPO recovery point. For lean it is the last BARRIER, never the last write
HITL human in the loop: a write that reaches the workspace from a UI rather than from the agent	the inbox the queue of UI writes waiting to be adopted. Entries name an object and its ETag, never a manifest edit	the window the barrier-in-progress token, in the SAME cell as the inbox — carrying a deadline, so it cannot wedge	epoch-validated checked PER REQUEST against the cell's current epoch, so a deposed worker's write is refused